



NeverStop

Operators in Java

Unit 4 • Lecture 1



NEVER STOP

■ **Class:** 9 ICSE

■ **Subject:** Computer Applications

■■■ **By:** Sourabh Gorai

Topics Covered

- ✦ What is an Operator & Operand?
- ✦ Unary • Binary • Ternary Operators
- ✦ Arithmetic • Relational • Logical • Assignment
- ✦ Increment • Decrement • Shorthand Operators
- ✦ Memory Tricks • Practice Questions

1. What is an Operator?

An **operator** is a special symbol (or keyword) that tells the computer to perform a specific operation on one or more values. Think of it like a mathematical sign — just as + means add, operators in Java give instructions to the program.

Real-Life Analogy

■ Imagine you're at a grocery store:

- Total = Price × Quantity
- Here × is the operator, and Price & Quantity are the operands.
- The operator 'operates' on the operands to give a result.

An **operand** is the value or variable on which the operator acts. In the expression 5 + 3, the numbers 5 and 3 are operands, and + is the operator.

Java Code

```
int a = 10;
int b = 5;
int sum = a + b;    // a and b are operands, + is the operator
// sum now holds 15
```

2. Forms of Operators

Operators are classified by how many operands they work on. Java has three forms:

Form	No. of Operands	Example	Meaning
Unary	1	-a or ++x	Works on a single value
Binary	2	a + b	Works on two values
Ternary	3	a > b ? a : b	Works on three values (if-else shortcut)

2.1 Unary Operator

Works on **just one operand**. It can negate a value, flip a boolean, or increase/decrease a number by 1.

Java Code

```
int x = 5;
System.out.println(-x);    // Output: -5    (negation)
boolean flag = true;
System.out.println(!flag); // Output: false  (logical NOT)
```

2.2 Binary Operator

Works on **two operands**. This is the most common form. Addition, subtraction, comparison — all are binary.

Java Code

```
int a = 8, b = 3;
int result = a + b;      // Binary: two operands a and b
boolean check = a > b;   // Binary: compares a with b
```

2.3 Ternary Operator (? :)

The **only operator in Java that needs three operands**. It is a one-line shortcut for an if-else statement.

Syntax: `condition ? value_if_true : value_if_false`

Java Code

```
int a = 10, b = 20;
int max = (a > b) ? a : b; // if a>b then max=a, else max=b
System.out.println(max);  // Output: 20
```

Quick Tip

■ Read `a > b ? a : b` as:

- "Is a greater than b? If YES give a, if NO give b."

3. Types of Operators

3.1 Arithmetic Operators

Used to perform basic **mathematical calculations**. They work on numeric types (int, float, double, etc.).

Operator	Name	Example (a=10, b=3)	Result
+	Addition	a + b	13
-	Subtraction	a - b	7
*	Multiplication	a * b	30
/	Division	a / b	3 (integer division)
%	Modulus	a % b	1 (remainder)

Note on / and %

- In Java, 10 / 3 gives 3 (not 3.33) because both are integers.
- To get a decimal result, use: 10.0 / 3 which gives 3.3333
- Modulus (%) gives the REMAINDER: 10 % 3 = 1
Think of it as: $10 = 3 \times 3 + 1 \rightarrow$ remainder is 1

Java Code

```
int a = 10, b = 3;
System.out.println(a + b); // 13
System.out.println(a - b); // 7
System.out.println(a * b); // 30
System.out.println(a / b); // 3 (integer division)
System.out.println(a % b); // 1 (remainder)
```

3.2 Relational (Comparison) Operators

Used to **compare two values**. The result is always a **boolean** — either **true** or **false**. They are widely used in if-statements and loops.

Operator	Meaning	Example (a=5, b=8)	Result
==	Equal to	a == b	false
!=	Not equal to	a != b	true
>	Greater than	a > b	false
<	Less than	a < b	true
>=	Greater than or equal	a >= 5	true

<=	Less than or equal	b <= 8	true
----	--------------------	--------	------

Common Mistake — = vs ==

- = is the ASSIGNMENT operator → stores a value
- == is the RELATIONAL operator → checks if two values are equal
- WRONG: if(a = 5) → this assigns 5 to a, not a comparison!
- CORRECT: if(a == 5) → this checks if a is equal to 5

Java Code

```
int a = 5, b = 8;
System.out.println(a == b); // false
System.out.println(a != b); // true
System.out.println(a < b);  // true
System.out.println(a >= 5); // true
```

3.3 Logical Operators

Used to **combine multiple conditions**. Like Relational operators, they also return a **boolean** result. Extremely useful in if-else and while-loop conditions.

Operator	Name	Result	Example
&&	AND	true only if BOTH conditions are true	a>0 && b>0
	OR	true if AT LEAST ONE condition is true	a>0 b>0
!	NOT	reverses the boolean value	!(a>0)

Truth Table for && and ||

A	B	A && B	A B
true	true	true	true
true	false	false	true
false	true	false	true
false	false	false	false

Java Code

```
int age = 15;
boolean hasID = true;
// Both must be true:
System.out.println(age >= 18 && hasID); // false
// At least one must be true:
System.out.println(age >= 18 || hasID); // true
// Reverse:
System.out.println(!(age >= 18)); // true
```

3.4 Assignment Operator

The assignment operator **=** stores a value into a variable. It always works **right to left** — the value on the right is placed into the variable on the left.

Java Code

```
int x;
x = 10; // 10 is stored into x
int y = x + 5; // x+5 (=15) is stored into y
x = x + 1; // old value of x (10) + 1 = 11, stored back in x
```

3.5 Increment and Decrement Operators

These are **unary operators** that add or subtract 1 from a variable. They come in two flavours: **prefix** (before the variable) and **postfix** (after the variable).

Operator	Name	Effect	Example
<code>++x</code>	Pre-increment	Increment FIRST, then use	<code>y = ++x</code>
<code>x++</code>	Post-increment	Use FIRST, then increment	<code>y = x++</code>
<code>--x</code>	Pre-decrement	Decrement FIRST, then use	<code>y = --x</code>
<code>x--</code>	Post-decrement	Use FIRST, then decrement	<code>y = x--</code>



Java Code

```
int a = 5;
int b = ++a; // a becomes 6 first, then b = 6
// Now a=6, b=6

int x = 5;
int y = x++; // y = 5 first, then x becomes 6
// Now x=6, y=5
```

3.6 Shorthand (Compound Assignment) Operators

Shorthand operators **combine an arithmetic operation with assignment**. They make code shorter and easier to read. Instead of writing `x = x + 5`, you write `x += 5` — same result!

Shorthand	Equivalent to	Example (x=10)	Result
<code>x += 5</code>	<code>x = x + 5</code>	<code>x += 5</code>	<code>x = 15</code>
<code>x -= 3</code>	<code>x = x - 3</code>	<code>x -= 3</code>	<code>x = 7</code>
<code>x *= 2</code>	<code>x = x * 2</code>	<code>x *= 2</code>	<code>x = 20</code>
<code>x /= 4</code>	<code>x = x / 4</code>	<code>x /= 4</code>	<code>x = 2</code>
<code>x %= 3</code>	<code>x = x % 3</code>	<code>x %= 3</code>	<code>x = 1</code>

Java Code

```
int x = 10;
x += 5;    // x = 15
x -= 3;    // x = 12
x *= 2;    // x = 24
x /= 4;    // x = 6
x %= 4;    // x = 2
System.out.println(x); // Output: 2
```

Quick Summary — All Operators at a Glance

Type	Operators	Returns
Arithmetic	<code>+</code> <code>-</code> <code>*</code> <code>/</code> <code>%</code>	Number
Relational	<code>==</code> <code>!=</code> <code>></code> <code><</code> <code>>=</code> <code><=</code>	boolean
Logical	<code>&&</code> <code> </code> <code>!</code>	boolean
Assignment	<code>=</code>	—
Increment	<code>++</code> (prefix & postfix)	Number
Decrement	<code>--</code> (prefix & postfix)	Number
Shorthand	<code>+=</code> <code>-=</code> <code>*=</code> <code>/=</code> <code>%=</code>	—
Ternary	<code>?:</code>	Any type

Memory Tricks

■ Memory Tricks

1. BODMAS for Arithmetic → Java follows the same precedence: () first, then * / %, then + -
2. = vs == : ONE equal (=) STORES, TWO equals (==) COMPARES. Think: '==' needs two to compare!
3. && = both hands needed (AND needs both to be true) | || = one hand is enough (OR needs just one)
4. PRE vs POST increment: PRE (++x) → 'Prepare first, then act' | POST (x++) → 'Act first, prepare later'
5. Modulus (%) → Think DIVISION REMAINDER: $10 \% 3 = 1$ because $10 \div 3 = 3$ remainder 1
6. Ternary = 3 parts: `CONDITION ? YES_VALUE : NO_VALUE` → like asking a question with two answers
7. Shorthand += means 'add to self': `x += 5` reads as 'x, add 5 to yourself'
8. URALTS mnemonic → Unary, Relational, Arithmetic, Logical, Ternary, Shorthand (types to remember)

Practice Questions

—■ Practice Questions

- Q1.** What is the difference between an operator and an operand? Give one example each.
- Q2.** Name the three forms of operators in Java. Give one example of each form.
- Q3.** What will be the output? `int a=7, b=2; System.out.println(a%b);`
- Q4.** Differentiate between = and == operators with an example.
- Q5.** What is the output? `int x=5; System.out.println(x++); System.out.println(x);`
- Q6.** What is the output? `int x=5; System.out.println(++x); System.out.println(x);`
- Q7.** Write the equivalent long form of: `x *= 3;` (where `x = 4`)
- Q8.** Evaluate: `(3 > 2) && (5 < 4)` → What boolean value does this return? Why?
- Q9.** Write a Java statement using the ternary operator to find the smaller of two numbers a and b.
- Q10** What will `10/4` give in Java? How is it different from `10.0/4`?
- Q11** Convert to shorthand: `a = a - 7;` and `b = b * 3;`
- Q12** Is `!true` equal to `false`? Write a Java statement to prove it.